

Лабораторная работа 5 - Сборка C-проектов (Make, Meson, CMake)

Цель работы. познакомиться с системами сборки (GNU Make, Meson, CMake) и организацией проектов на языке C. Научиться создавать проекты с различной структурой директорий (плоской и иерархической), подключать внешние библиотеки и готовить сборочные скрипты для разных инструментов. В рамках работы предлагается реализовать **8 небольших проектов** на C, тематически разделённых на две группы: четыре проекта с плоской структурой файлов и четыре проекта со стандартной иерархией каталогов. Для каждого проекта приводится сборка с помощью **GNU Make**, а также эквивалентные конфигурации **Meson** и **CMake**. Кроме того, приведены теоретические сведения: основы написания Makefile, файлов meson.build и CMakeLists.txt, различия между плоской и иерархической структурами проектов, подключение сторонних библиотек (SDL2, GSL, libcurl и др.), а также советы по оформлению проектов в условиях командной разработки.

Теоретическая справка

Системы сборки: GNU Make, Meson, CMake

GNU Make. GNU Make – одна из самых старых и распространённых систем автоматизации сборки C/C++ проектов. Make использует специальный файл правил (Makefile), в котором описано, какие файлы и как необходимо компилировать и компоновать. Основываясь на правилах и отслеживая временные метки файлов, утилита make сама решает, какие части программы нужно перекомпилировать при изменениях исходников. Таким образом, Make позволяет избежать лишней перекомпиляции и ускоряет сборку крупных проектов. В Makefile задаются *цели* (targets) – обычно имена итоговых исполняемых файлов или промежуточных объектов – и их *зависимости* (dependencies) – исходные файлы или другие цели. Для каждой цели указывается *рецепт* – команды (обычно вызов компилятора/компоновщика) для создания этой цели из зависимостей. Например, цель для объекта module.o зависит от module.c и компилируется командой \$(CC) -c module.c -o module.o, а итоговая цель программы зависит от всех .o файлов и компонуется командой линковщика. Make также поддерживает *фонии* (phony targets) вроде clean для очистки сгенерированных файлов, переменные для флагов компиляции, шаблоны правил и включает механизм include для автоматического отслеживания заголовочных файлов (через генерацию .d зависимостей). **Итог.** Makefile предоставляет разработчику гибкий способ описать процесс сборки проекта, но требует внимательного ручного прописывания правил и зависимостей.

Meson. Meson – современная система сборки, ориентированная на высокую скорость и удобство для разработчика. В отличие от Make, Meson не использует процедуры, описанные на языке shell; вместо этого сборка описывается на языке Meson (DSL), который **не** является Turing-полным скриптовым языком, а представляет декларативный формат. Meson нацелен на минимизацию времени, которое программист тратит на написание и отладку файлов сборки. Meson-проект описывается файлом meson.build, в котором указываются проекты, исходники и зависимости. Meson сам генерирует необходимые файлы для выбранной нижележащей системы сборки (по умолчанию Ninja, но может также генерировать Makefile, Visual Studio проекты и др.). Главное преимущество Meson – простота и автоматизация: подключение внешних библиотек требует лишь указать dependency('имя'), Meson найдет библиотеку (через pkg-config или иными способами) и сам добавит нужные флаги компиляции и линковки. Например, чтобы использовать библиотеку zlib в Meson, достаточно написать:

```
zlib_dep = dependency('zlib')           # поиск внешней библиотеки zlib
exe = executable('prog', 'main.c',      # сборка исполняемого файла
                dependencies : zlib_dep) # слинковать с zlib
```

Meson автоматически определит флаги компилятора и линковщика для zlib (как пути включения заголовков, имена .so/.lib и т.д.). Meson считается более понятным и «человеко-дружелюбным» инструментом, чем ручное написание Makefile: формат meson.build более декларативный и лаконичный. Кроме того, Meson кроссплатформенный и поддерживает многие языки (C, C++, D, Fortran, Java и т.д.). Для сборки Meson-проекта обычно используются две команды: meson setup <builddir> (генерация файлов сборки в папке builddir) и затем meson compile -C <builddir> (запуск сборки).

CMake. CMake – это не сам компилятор и не утилита сборки в узком смысле, а **генератор проектов** (build system generator). Файлы CMakeLists.txt описывают структуру проекта (исходники, цели, зависимости) на специальном языке (синтаксис CMake). При конфигурации (cmake .) CMake генерирует файлы для конкретной системы сборки – например, Makefile для Unix, проект Visual Studio для Windows, файлы Ninja и др.. Таким образом, CMake является прослойкой, которая позволяет с единых исходных настроек получить переносимый проект. Преимущество CMake в том, что один CMakeLists.txt может сгенерировать сборочные файлы для разных платформ и инструментов (Make, Ninja, Xcode, Visual Studio и пр.). На больших кроссплатформенных проектах CMake стал *де-факто* стандартом. Однако язык CMake довольно специфичен и содержит множество директив; неосторожное обращение с ним может приводить к трудно отлаживаемым скриптам. Тем не менее, современные версии CMake улучшились, добавив так называемые “импортируемые цели” (импортированные библиотеки) и упрощённую модульную систему поиска библиотек (find_package). Типичный CMakeLists.txt начинается с указания минимальной версии cmake_minimum_required и имени проекта project(), затем определяются цели через add_executable или add_library, указываются исходники, заголовочные директории (include_directories() или target_include_directories), а также зависимости. Для подключения библиотек используется команда find_package(), которая ищет установленную библиотеку и определяет для неё специальные targets. Например, find_package(GSL REQUIRED) найдёт GSL и предоставит импортируемые цели GSL::gsl и GSL::gslcblas, которые затем привязываются к исполняемому файлу через target_link_libraries(myexe GSL::gsl GSL::gslcblas). CMake тем самым позволяет не указывать вручную пути и флаги для известных библиотек, если для них предусмотрены модули Find<Name> или config-файлы.

Вывод. GNU Make требует ручного написания правил сборки, Meson предлагает более автоматизированный и быстрый подход, а CMake генерирует сборочные скрипты для разных систем. Знание всех трёх инструментов полезно: Make демонстрирует базовые принципы автоматизации сборки, Meson показывает современные практики, а CMake нужен для понимания кроссплатформенной сборки крупных проектов.

Структура проектов: плоская vs. иерархическая

Организация файлов в проекте может значительно влиять на удобство разработки и совместной работы. Существуют два распространённых подхода к структурированию исходников:

- **«Плоская» структура.** Все исходные файлы (.c) и заголовки (.h), а также файл сборки (Makefile и др.) находятся в одном каталоге проекта. Эта схема характерна для небольших проектов, она максимально проста и позволяет быстро ориентироваться, когда файлов мало. Нет вложенных модулей или пакетов – вся логика организована либо в отдельных файлах по функционалу, либо даже внутри одного-двух исходников. Преимущество плоской структуры – минимальные накладные расходы на организацию; ничего лишнего, только необходимые файлы. Однако по мере роста числа файлов такая структура становится громоздкой: трудно группировать связанные части проекта, можно случайно получить конфликт имён файлов, смешиваются файлы разного назначения (исходники, объекты, исполняемые файлы, ресурсы) в одном месте.
- **«Иерархическая» структура (модульная).** Проект разделяется на подпапки по смысловым модулям или типам файлов. Типичный вариант – директории **src/** (исходники .c), **include/** (заголовки .h), **obj/** (объектные .o файлы), **bin/** (собранные исполняемые файлы), **lib/** (внешние библиотеки или сторонний код), **docs/** (документация) и т.д. В рамках модульного подхода крупные проекты могут иметь поддиректории в src/ по подсистемам, каждую со своими исходниками и заголовками. Иерархическая структура облегчает навигацию в проекте: файлы разного типа и назначения разложены по своим местам, код разбит на компоненты. Такой подход упрощает масштабирование – новую функциональность добавляют как новый модуль (новая папка в src/, свой заголовок в include/). **Минус** – небольшому проекту нет смысла заводить много папок; избыточная структура усложнит сборку. Поэтому для учебных или маленьких задач лучше начать с простой схемы, а при усложнении перейти к иерархии.

Важно понимать, что выбор структуры – **не строгий стандарт**, а договорённость в команде. Тем не менее, в индустрии сложились типовые шаблоны: практически любой крупный C-проект имеет разделение на src/include, папки для build-артефактов и т.д. Ниже в практической части мы продемонстрируем оба подхода на примерах.

Подключение сторонних библиотек

Реальные проекты редко ограничиваются стандартной библиотекой языка C – часто требуется использовать внешние библиотеки: графические (например, SDL2), научные (GSL – GNU Scientific Library), сетевые (libcurl для HTTP, OpenSSL для шифрования и т.д.), парсеры форматов (XML, JSON) и многие другие. Вопрос – **как правильно подключить и собрать проект с такой библиотекой?**

В Си внешняя библиотека обычно состоит из заголовочных файлов (.h) и скомпилированных объектных модулей (статическая .a или динамическая .so/.dll). Чтобы использовать библиотеку, нужно:

1. **Добавить пути к заголовкам** при компиляции (-I/path/to/include или через переменные окружения/файлы конфигурации).
2. **Указать сами библиотеки при компоновке** (-L/path/to/lib -l<name>).

Ручное указание путей не всегда удобно, особенно если библиотека установлена системой. Здесь на помощь приходит утилита **pkg-config** – стандартизованный способ узнать у системы, где находятся файлы конкретной библиотеки и какие флаги нужны. Многие библиотеки поставляют .pc-файл для pkg-config. Например, для SDL2 пакет предоставляет информацию, что для компиляции нужны флаги \$(pkg-config --cflags sdl2) (вернут включение нужных директорий, определение макросов) а для линковки – \$(pkg-config --libs sdl2). В Makefile можно использовать вызов pkg-config прямо в определении переменных:

```
CFLAGS += $(shell pkg-config --cflags sdl2)
LDLIBS += $(shell pkg-config --libs sdl2)
```

Таким образом, **Makefile** упрощается – не надо вручную прописывать -I/usr/include/SDL2 и список .so библиотек SDL2, pkg-config подставит их автоматически. Аналогично, pkg-config помогает с библиотекой GSL (pkg-config --libs gsl вернет, к примеру, -lgsl -lgslcblas -lm), с libcurl (pkg-config --libs libcurl -> -lcurl плюс, возможно, зависимые библиотеки), с XML-парсером libxml2 (pkg-config --cflags --libs libxml-2.0 вернет набор флагов для включения <libxml/parser.h> и ссылки на -lxml2 и др.).

Meson в этом плане ещё удобнее: достаточно в meson.build написать dependency('SDL2') или dependency('libcurl') – Meson сам воспользуется pkg-config или встроенными механизмами и найдет все необходимые флаги. Разработчику не нужно явно прописывать опции компиляции/линковки – Meson добавит их “под капотом”.

CMake тоже поддерживает pkg-config (через модуль FindPkgConfig) и имеет модули FindSDL2, FindGSL и пр. Как правило, подключение библиотеки в CMake делается так:

```
find_package(Name REQUIRED)
target_link_libraries(myexe Name::Name)
```

где Name::Name – импортированный target библиотеки. Как отмечалось выше, find_package находит библиотеку и определяет её интерфейс (включая пути и сами .lib/.so). Например, find_package(GSL) предоставит цели GSL::gsl, GSL::gslcblas; find_package(SDL2) – цель SDL2::SDL2 (в новых версиях SDL2 есть официальный config для CMake). Если же готового модуля нет, можно воспользоваться pkg_check_modules (для pkg-config) либо указать пути вручную.

Пример. в проектах далее мы покажем, как подключать:

- **SDL2** (графическая библиотека): для отрисовки окон и 2D-графики,
- **GSL** (научная библиотека): для сложных вычислений (например, статистика, линалг и пр.),
- **pthread** (POSIX Threads): для работы с потоками (понадобится при расширении сетевого проекта),
- **libcurl** (библиотека для HTTP и URL): для загрузки веб-страниц,
- **cJSON** или **libxml2**: для разбора JSON и XML структур.

Каждый из этих примеров даст опыт работы с подключаемыми библиотеками. **Важно.** при использовании сторонних библиотек убедитесь, что они установлены в системе (например, через пакетный менеджер) и доступен их .h и .so/.a. В противном случае сборка не найдёт заголовки или выдаст ошибки undefined reference при линковке.

Оформление проектов в командной разработке

Когда над кодом работает команда, важно выработать единые подходы к структуре и стилю. Вот несколько рекомендаций:

- **Система контроля версий (VCS).** Используйте Git или аналог для хранения кода. Структура репозитория должна повторять структуру проекта. Игнорируйте в VCS временные файлы сборки (obj/, исполняемые файлы, сгенерированные артефакты) – для этого в репозитории добавляют файл `.gitignore`.
- **Единый стиль кода.** В команде договоритесь о стиле оформления (отступы, именование переменных, расположение { } и т.д.). Желательно настроить автоформатирование (например, файл `.clang-format` как в open-source проектах) и линтер. Это устранил различия в стиле у разных разработчиков.
- **Документирование и комментарии.** Обеспечьте, чтобы каждый модуль имел краткое описание (в начале файла или в README). Комментарии в коде помогают понять логику, особенно когда над проектом работают несколько человек.
- **Модульность и разграничение ответственности.** Иерархическая структура помогает распределить задачи – каждый разработчик может работать над своим модулем (своей папкой) с минимальными пересечениями. Например, один пишет модуль графики, другой – сетевой модуль. На этапе интеграции проект собирается из модулей через общие заголовки и интерфейсы.
- **Сборочные скрипты и среды.** Если в команде кто-то работает под Windows, а кто-то под Linux, CMake или Meson помогут унифицировать сборку. Создайте инструкции по сборке проекта в README, чтобы новый участник команды мог быстро запустить проект.
- **Код-ревью и тестирование.** При совместной работе полезно внедрить практику взаимного обзора кода (code review) перед слиянием изменений. Также стоит вести набор тестов (можно автоматизировать сборкой `make test` или аналогом в Meson/CMake), чтобы убедиться, что новый код не ломает существующий функционал.

Следуя этим принципам, команда сможет поддерживать проект в чистоте и порядке: понятная структура + стройный процесс сборки + согласованный код = эффективная командная разработка.

Практические проекты

Ниже представлены восемь учебных проектов на языке C. Первые четыре проекта используют **плоскую** структуру директорий и базовые библиотеки; следующие четыре – **иерархическую** структуру (с разделением на `src/`, `include/` и `pr.`) и подключение дополнительных модулей. Для **каждого проекта** описывается его назначение, структура файлов, приводятся фрагменты кода, а также примеры сборочных файлов: **Makefile**, **meson.build** и **CMakeLists.txt**. Вам предлагается изучить и сопоставить эти файлы, чтобы понять, как один и тот же проект выражается в разных системах сборки.

Проекты с плоской структурой каталогов

Все файлы исходного кода и сборки находятся в одном каталоге. Отсутствует разграничение на `src/include` – заголовочные файлы лежат рядом с `.c`. Объектные файлы (`.o`) и исполняемые будут генерироваться либо в тот же каталог, либо в поддиректории, если указано (в простых Makefile ниже они остаются в текущем каталоге). Такая структура упрощает навигацию для маленького проекта.

Проект 1: Работа с файлами и структурами данных

Описание. Первый проект посвящён чтению и записи файлов, а также использованию структур данных в C. В качестве примера можно реализовать программу, которая читает данные из текстового файла (например, список студентов с оценками) и загружает их в структуру (например, массив структур `Student`). Затем программа может выполнять базовую обработку: сортировать записи, вычислять среднее значение, сохранять результаты в новый файл. Цель – закрепить навыки работы с файловым вводом/выводом (`fopen`, `fscanf/fprintf`, `fgets` и т.д.) и определением структур (`struct`) и их использованием.

Структура директорий (плоская). Все файлы проекта лежат в одном каталоге `proj1/`:

```

proj1/
├─ main.c          # основная логика программы (чтение файла, вызов обработки)
├─ data.c          # функции для работы с структурами данных (например, сортировка)
├─ data.h          # определение структуры данных (например, struct Student) и прототипы
└─ функций
├─ Makefile        # сборка с помощью GNU Make
├─ meson.build      # сборка с помощью Meson
└─ CMakeLists.txt  # сборка с помощью CMake

```

Примеры исходных файлов:

data.h – заголовочный файл с определением структуры и функций:

```

// data.h
#ifndef DATA_H
#define DATA_H

#include <stdio.h>

typedef struct {
    char name[50];
    int scores[5];
    double average;
} Student;

// Прототипы функций
int loadStudents(const char *filename, Student *arr, size_t max);
void sortByAverage(Student *arr, size_t count);
void saveReport(const char *filename, Student *arr, size_t count);

#endif

```

data.c – реализация функций работы с Student:

```

// data.c
#include "data.h"
#include <string.h>
#include <stdlib.h>

int loadStudents(const char *filename, Student *arr, size_t max) {
    FILE *f = fopen(filename, "r");
    if (!f) return -1;
    size_t count = 0;
    while (count < max && fscanf(f, "%49[^,],%d,%d,%d,%d,%d\n",
        arr[count].name,
        &arr[count].scores[0], &arr[count].scores[1],
        &arr[count].scores[2], &arr[count].scores[3],
        &arr[count].scores[4]) == 6) {
        // вычисляем средний балл
        int sum = 0;
        for(int i=0; i<5; ++i) sum += arr[count].scores[i];
        arr[count].average = sum / 5.0;
        count++;
    }
    fclose(f);
}

```

```

    return count;
}

void sortByAverage(Student *arr, size_t count) {
    // простая сортировка пузырьком по average
    for(size_t i=0; i<count; ++i) {
        for(size_t j=0; j<count-i-1; ++j) {
            if(arr[j].average < arr[j+1].average) {
                Student tmp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = tmp;
            }
        }
    }
}

void saveReport(const char *filename, Student *arr, size_t count) {
    FILE *f = fopen(filename, "w");
    if (!f) return;
    fprintf(f, "Name,Average\n");
    for(size_t i=0; i<count; ++i) {
        fprintf(f, "%s,%.2f\n", arr[i].name, arr[i].average);
    }
    fclose(f);
}

```

main.c – основная функция, связывающая всё вместе:

```

// main.c
#include "data.h"
#include <stdio.h>

#define MAX_STUDENTS 100

int main(int argc, char *argv[]) {
    if (argc < 3) {
        fprintf(stderr, "Usage: %s <input_file> <output_file>\n", argv[0]);
        return 1;
    }
    Student students[MAX_STUDENTS];
    int count = loadStudents(argv[1], students, MAX_STUDENTS);
    if (count < 0) {
        perror("Error opening input file");
        return 1;
    }
    sortByAverage(students, count);
    saveReport(argv[2], students, count);
    printf("Processed %d students. Report saved to %s\n", count, argv[2]);
    return 0;
}

```

Makefile (плоская структура). Для сборки этого проекта Makefile достаточно простой. Компилируем все .c в объектные файлы и линкуем в исполняемый файл, назовём его, например, proj1.

Makefile for Project 1

```

CC := gcc
CFLAGS := -Wall -Wextra -std=c11 -O2  # флаги компиляции
LDFLAGS :=
LDLIBS := -lm      # связываем с math lib, если нужны математические функции

# Список исходников и целевой исполняемый файл
SRC := main.c data.c
OBJ := $(SRC:.c=.o)
TARGET := proj1

all: $(TARGET)

$(TARGET): $(OBJ)
    $(CC) $(LDFLAGS) $(OBJ) -o $@ $(LDLIBS)

# Правило для создания .o из .c (можно опустить, GNU Make имеет неявное правило)
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    $(RM) $(OBJ) $(TARGET)

```

Замечания: Здесь мы указали `LDLIBS := -lm` на случай, если код использует функции `sqrt`, `pow` и прочие из `<math.h>` (мат. библиотека). В данном проекте, правда, `math.h` может и не понадобиться, но это шаблон. В плоской структуре все `.o` будут созданы в том же каталоге, `clean` их удаляет.

Meson (meson.build). Эквивалентная сборка через Meson – одна строка для исполняемого файла. Meson сам распознает, какие файлы нужно скомпилировать, и определит зависимости.

```
project('proj1-files-and-structs', 'c', version: '1.0')
```

```

executable('proj1', ['main.c', 'data.c'],
    include_directories: '.', # текущий каталог содержит data.h
    install: true)

```

Здесь мы указываем `include_directories: '.'` чтобы Meson знал, где искать `data.h` (в текущем каталоге). Опция `install: true` не обязательна, но означает, что при `meson install` бинарник будет установлен в систему. Для данного учебного примера это несущественно.

В Meson внешние библиотеки не используются, поэтому нет поля `dependencies` – достаточно стандартных библиотек. Meson по умолчанию использует компилятор `gcc/clang` (в зависимости от системы) с флагами оптимизации и отладки, ничего дополнительного указывать не нужно.

CMake (CMakeLists.txt). Пример CMakeLists для этого проекта:

```

cmake_minimum_required(VERSION 3.12)
project(proj1-files-and-structs C)  # имя проекта и язык

set(SRC_LIST main.c data.c)        # список исходников
add_executable(proj1 ${SRC_LIST})   # создаём исполняемый файл из исходников
target_include_directories(proj1 PUBLIC ${CMAKE_CURRENT_SOURCE_DIR})
# ^ включаем текущий каталог в пути для заголовочных файлов (чтобы data.h находился)
target_link_libraries(proj1 PUBLIC m)
# ^ линковка с математической библиотекой libm (аналог -lm)

```

Обратите внимание: в CMake для линковки стандартной математической библиотеки используется таргет `m` (или

можно написать `-lm` в `target_link_libraries`, но принято указывать именно `m`). Директория с исходниками добавлена в `include_directories`, чтобы `#include "data.h"` нашёл файл.

Проект 2: Сокетное взаимодействие (сетевое приложение)

Описание. Этот проект знакомит вас с основами сетевого программирования на C (берклиевские сокеты). Можно реализовать простое клиент-сервер приложение. Например:

- Сервер (консольная программа), которая прослушивает определённый TCP порт и при подключении клиента принимает от него сообщения и отправляет обратно (эхо-сервер).
- Клиент, который подключается к серверу, отправляет текстовую строку и получает обратно ответ.

Либо можно сделать одностороннее взаимодействие, например, программа-клиент скачивает страницу по протоколу HTTP (то есть устанавливает сокет-соединение с web-сервером на 80 порт и посылает HTTP-запрос). В учебных целях достаточно эхо-сервера и клиента. Вы познакомитесь с системными вызовами `socket()`, `bind()`, `listen()`, `accept()` на стороне сервера и `connect()` на стороне клиента, а также с передачей данных через `send()/recv()` (или `read()/write()`). Работа с сокетами требует понимания сетевых адресов (структура `sockaddr_in`), конвертации байтовых порядков (`htonl`, `htons`), поэтому проект даёт практику системного программирования.

Структура (плоская). Предположим, сделаем два исполняемых файла – сервер и клиент – в рамках одного проекта. Для простоты всё же можно объединить в одно приложение с разными режимами (например, опция командной строки `--server` или `--client`), но чаще это отдельные программы. В рамках одной директории создадим соответственно два `main` файла. Структура `proj2/`:

```
proj2/
├── server.c      # исходник сервера
├── client.c      # исходник клиента
├── net_utils.c   # вспомогательные функции, общие для клиента и сервера
├── net_utils.h   # заголовок с прототипами общих сетевых функций
├── Makefile
├── meson.build
└── CMakeLists.txt
```

Пример кода (фрагменты):

`net_utils.h` – общие объявления:

```
// net_utils.h
#ifndef NET_UTILS_H
#define NET_UTILS_H

#include <netinet/in.h> // sockaddr_in

int create_server_socket(int port);
int create_client_socket(const char *host, int port);
ssize_t send_all(int sockfd, const void *buffer, size_t length);
ssize_t recv_all(int sockfd, void *buffer, size_t maxlen);

#endif
```

`net_utils.c` – реализация некоторых общих функций:

```
// net_utils.c
#include "net_utils.h"
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
```



```

int create_server_socket(int port) {
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("socket");
        return -1;
    }
    // Позволим повторно использовать адрес/порт:
    int opt = 1;
    setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    // Привязываем сокет к указанному порту на всех интерфейсах:
    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);
    addr.sin_addr.s_addr = INADDR_ANY;
    if (bind(sockfd, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("bind");
        close(sockfd);
        return -1;
    }
    if (listen(sockfd, 5) < 0) { // очередь на 5 подключений
        perror("listen");
        close(sockfd);
        return -1;
    }
    return sockfd;
}

int create_client_socket(const char *host, int port) {
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) { perror("socket"); return -1; }
    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);
    if (inet_pton(AF_INET, host, &addr.sin_addr) <= 0) {
        fprintf(stderr, "Invalid address: %s\n", host);
        close(sockfd);
        return -1;
    }
    if (connect(sockfd, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("connect");
        close(sockfd);
        return -1;
    }
    return sockfd;
}

// send_all и recv_all пытаются отправить/получить все данные заданной длины
ssize_t send_all(int sockfd, const void *buffer, size_t length) {
    const char *ptr = buffer;
    size_t remaining = length;
    while (remaining > 0) {
        ssize_t sent = send(sockfd, ptr, remaining, 0);

```

```

        if (sent <= 0) return sent;
        remaining -= sent;
        ptr += sent;
    }
    return length;
}

ssize_t recv_all(int sockfd, void *buffer, size_t maxlen) {
    char *ptr = buffer;
    size_t total = 0;
    while (total < maxlen) {
        ssize_t recvd = recv(sockfd, ptr + total, maxlen - total, 0);
        if (recvd <= 0) {
            return (recvd == 0) ? total : -1;
        }
        total += recvd;
        // остановимся, если получили меньше, чем запрошено (значит, данных больше нет)
        if ((size_t)recvd < maxlen - total) break;
    }
    return total;
}

```

server.c – основной код сервера:

```

// server.c
#include "net_utils.h"
#include <stdio.h>
#include <unistd.h>

#define PORT 8080

int main() {
    int server_fd = create_server_socket(PORT);
    if (server_fd < 0) {
        return 1;
    }
    printf("Server listening on port %d...\n", PORT);
    while (1) {
        int client_fd = accept(server_fd, NULL, NULL);
        if (client_fd < 0) {
            perror("accept");
            break;
        }
        printf("Client connected, echoing data...\n");
        char buf[1024];
        ssize_t n;
        // эхо-цикл: читаем от клиента и отправляем обратно
        while ((n = recv_all(client_fd, buf, sizeof(buf))) > 0) {
            send_all(client_fd, buf, n);
        }
        close(client_fd);
        printf("Client disconnected.\n");
    }
    close(server_fd);
}

```

```

    return 0;
}

```

client.c – основной код клиента:

```

// client.c
#include "net_utils.h"
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#define SERVER_ADDR "127.0.0.1"
#define PORT 8080

int main() {
    int sock = create_client_socket(SERVER_ADDR, PORT);
    if (sock < 0) {
        return 1;
    }
    printf("Connected to server %s:%d\n", SERVER_ADDR, PORT);
    char msg[256];
    printf("Enter message: ");
    if (!fgets(msg, sizeof(msg), stdin)) {
        close(sock);
        return 0;
    }
    size_t len = strlen(msg);
    send_all(sock, msg, len);
    char reply[256] = {0};
    ssize_t received = recv_all(sock, reply, len);
    if (received >= 0) {
        printf("Server replied: %.*s\n", (int)received, reply);
    } else {
        printf("Error receiving response\n");
    }
    close(sock);
    return 0;
}

```

Эта пара программ реализует эхо-клиент и сервер. Сервер многократно принимает новых клиентов, обслуживая по одному за раз (в расширенной версии можно сделать многопоточное обслуживание – см. проект 7). Клиент отправляет одну строчку и ждёт эхо-ответа.

Makefile. В этом проекте хотим получить два исполняемых файла: server и client. Соответственно, Makefile будет компилировать server.c, client.c и общие net_utils.c:

```

# Makefile for Project 2 (Sockets)
CC := gcc
CFLAGS := -Wall -Wextra -std=c11 -O2
LDFLAGS :=
LDLIBS :=          # на Linux сокет-функции находятся в libc, отдельные -lnet/-lnsl обычно не
                   ↪ нужны

# Исполняемые файлы и их соответствующие исходники
TARGETS := server client

```

```

OBJJS_server := server.o net_utils.o
OBJJS_client := client.o net_utils.o

all: $(TARGETS)

server: $(OBJJS_server)
    $(CC) $(LDFLAGS) $(OBJJS_server) -o server $(LDLIBS)

client: $(OBJJS_client)
    $(CC) $(LDFLAGS) $(OBJJS_client) -o client $(LDLIBS)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    $(RM) $(TARGETS) *.o

```

Обратите внимание, что здесь у нас две цели: `server` и `client`. Мы указали, что для линковки никаких специальных библиотек не нужно (`LDLIBS` пустой) – базовые сокет не требуют дополнительных либ (функции `socket`, `bind`, etc. находятся в стандартной библиотеке). В некоторых случаях может понадобиться `-pthread` (если бы использовали потоки) или `-lnsl` на старых UNIX-системах, но для Linux GCC обычно нет. Если бы код содержал `getaddrinfo/gethostbyname`, возможно, потребовалась бы линковка с `-lanl`, но не будем усложнять.

Meson (`meson.build`). Meson позволяет легко определить несколько исполняемых файлов. Мы также воспользуемся тем, что у Meson есть функция группировки исходников.

```

project('proj2-sockets', 'c', version: '1.0')
# Определяем общие исходники и объекты
sources = ['net_utils.c']

executable('server', sources + ['server.c'],
    include_directories: ['.'])
executable('client', sources + ['client.c'],
    include_directories: ['.'])

```

Здесь мы объявили список `sources`, содержащий `net_utils.c` (и по умолчанию Meson сам разберётся, что нужен `net_utils.h` из текущей директории для его компиляции). Затем два раза вызвали `executable` для разных целей. Meson сам позаботится, чтобы `net_utils.c` был скомпилирован отдельно для каждого исполняемого (хотя он может выполнить компиляцию один раз и просто слинковать оба, но, скорее всего, он скомпилирует две копии – изоляция целей).

CMake (`CMakeLists.txt`). В CMake для двух исполняемых также пишем два `add_executable`:

```

cmake_minimum_required(VERSION 3.12)
project(proj2-sockets C)

add_executable(server server.c net_utils.c)
add_executable(client client.c net_utils.c)

# Можно вынести общие флаги в переменные, например, для всех:
set_target_properties(server client PROPERTIES C_STANDARD 11 C_STANDARD_REQUIRED YES)

# Включаем текущий каталог для поиска net_utils.h
target_include_directories(server PRIVATE ${CMAKE_CURRENT_SOURCE_DIR})
target_include_directories(client PRIVATE ${CMAKE_CURRENT_SOURCE_DIR})

```

Здесь мы прямо в `add_executable` перечислили нужные файлы. `set_target_properties` — это сразу для двух целей устанавливаем стандарт C11 (опционально). Указали `include_directories`, чтобы нашли заголовки.

Проект 3: Парсинг HTML-страниц

Описание. Третий проект демонстрирует обработку текстовых данных сложного формата – HTML. Задача: взять HTML-файл (или скачать веб-страницу) и извлечь из него определённую информацию. Например, программа может вытащить содержимое тегов `<title>` или все ссылки `<a href="...">` из HTML-файла. Это учит работать со строками, искать подстроки, возможно, использовать регулярные выражения или парсить по структуре тегов.

Для парсинга HTML есть два подхода:

1. **Простой (без внешних библиотек).** Обработать файл как текст, используя функции `<string.h>` (`strstr`, `strtok` и т.д.). Например, искать подстроку `"<title>"`, затем `"</title>"` и выделять между ними. Аналогично для ссылок искать `href="..."`
2. **С библиотеками.** Использовать готовые парсеры HTML/XML, например, **libxml2** (она умеет парсить HTML в DOM-дерево), или более простую библиотеку **Gumbo** (HTML parser in C). Также можно воспользоваться **libcurl** для скачивания страницы по URL, чтобы не сохранять её вручную.

В рамках лабораторной можно предложить сначала реализовать простейший парсер самостоятельно (вариант 1), а затем показать, как использовать библиотеку (вариант 2). Например, написать функцию `parse_title(const char *html, char *out_title)` которая находит в строке `html` начало `<title>` и конец `</title>` и копирует содержимое в буфер `out_title`. Для более сложных структур (ссылки, списки) код на C станет громоздким, поэтому познакомимся с `libxml2` или подобной библиотекой.

Структура (плоская). Предположим, у нас будет программа, которая умеет либо читать локальный HTML-файл, либо загружать URL (по выбору). Назовём её `htmlparser`. Структура `proj3/`:

```
proj3/
├─ main.c           # основная логика: чтение файла или скачивание, вызов парсера
├─ htmlutils.c      # функции для разбора HTML (самописные или обёртки вокруг libxml2)
├─ htmlutils.h
├─ Makefile
├─ meson.build
└─ CMakeLists.txt
```

Примеры кода:

`htmlutils.h` – объявление функций:

```
// htmlutils.h
#ifndef HTMLUTILS_H
#define HTMLUTILS_H

#include <stddef.h>

int extract_title(const char *html, char *out, size_t out_size);
// возможно, другие функции: extract_links, и т.д.

#endif
```

`htmlutils.c` – простая реализация парсинга тега `<title>` вручную:

```
// htmlutils.c
#include "htmlutils.h"
#include <string.h>

int extract_title(const char *html, char *out, size_t out_size) {
    const char *tag_start = "<title>";
    const char *tag_end = "</title>";
    char *pos1 = strstr(html, tag_start);
```

```

    if (!pos1) return -1;
    pos1 += strlen(tag_start);
    char *pos2 = strstr(pos1, tag_end);
    if (!pos2) return -1;
    size_t title_len = pos2 - pos1;
    if (title_len >= out_size) title_len = out_size - 1;
    memcpy(out, pos1, title_len);
    out[title_len] = '\\0';
    return 0;
}

```

Эта функция ищет в строке HTML начало и конец тега title и копирует содержимое. Она уязвима, если теги в разном регистре или есть пробелы, но для учебной задачи пойдёт.

main.c – основная программа:

```

// main.c
#include "htmlutils.h"
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

// Если определено USE_CURL, будем использовать libcurl для скачивания
#ifdef USE_CURL
#include <curl/curl.h>
struct Buffer { char *data; size_t size; };
size_t write_cb(void *contents, size_t size, size_t nmemb, void *userp) {
    size_t total = size * nmemb;
    struct Buffer *buf = (struct Buffer*) userp;
    char *ptr = realloc(buf->data, buf->size + total + 1);
    if(!ptr) return 0;
    buf->data = ptr;
    memcpy(&(buf->data[buf->size]), contents, total);
    buf->size += total;
    buf->data[buf->size] = '\\0';
    return total;
}
char *download_page(const char *url) {
    CURL *curl = curl_easy_init();
    if(!curl) return NULL;
    struct Buffer buf = { .data = malloc(1), .size = 0 };
    curl_easy_setopt(curl, CURLOPT_URL, url);
    curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, write_cb);
    curl_easy_setopt(curl, CURLOPT_WRITEDATA, &buf);
    CURLcode res = curl_easy_perform(curl);
    curl_easy_cleanup(curl);
    if(res != CURLE_OK) {
        free(buf.data);
        return NULL;
    }
    return buf.data;
}
#endif

```

```

int main(int argc, char *argv[]) {
    if(argc < 2) {
        fprintf(stderr, "Usage: %s <file_or_url>\n", argv[0]);
        return 1;
    }
    char *html_content = NULL;
    // Проверим, начинается ли аргумент с "http://"
    if(strncmp(argv[1], "http://", 7) == 0 || strncmp(argv[1], "https://", 8) == 0) {
#ifdef USE_CURL
        html_content = download_page(argv[1]);
        if(!html_content) {
            fprintf(stderr, "Failed to download %s\n", argv[1]);
            return 1;
        }
#else
        fprintf(stderr, "This program was built without libcurl support.\n");
        return 1;
#endif
    } else {
        // читаем локальный файл
        FILE *f = fopen(argv[1], "r");
        if(!f) { perror("fopen"); return 1; }
        fseek(f, 0, SEEK_END);
        long size = ftell(f);
        fseek(f, 0, SEEK_SET);
        html_content = malloc(size + 1);
        if(html_content) {
            fread(html_content, 1, size, f);
            html_content[size] = '\0';
        }
        fclose(f);
    }

    if(!html_content) {
        fprintf(stderr, "Could not load content.\n");
        return 1;
    }

    char title[256];
    if(extract_title(html_content, title, sizeof(title)) == 0) {
        printf("Title: %s\n", title);
    } else {
        printf("Title tag not found.\n");
    }

    free(html_content);
    return 0;
}

```

В этом коде реализована поддержка опционального использования **libcurl**: если при компиляции определить макрос `USE_CURL`, то будет включён код загрузки по URL. Это сделано, чтобы показать вам, что можно условно подключать функциональность (например, через флаг компиляции). Функция `download_page` использует `libcurl` API: инициализация, установка URL, указание колбэка для записи (наша `write_cb` накапливает данные в буфер), выполнение запроса

и завершение. Если libcurl не используется, программа ожидает файл на диске.

Makefile. Данный проект может быть собран с или без curl. Предположим, по умолчанию мы хотим собрать с поддержкой curl. В Linux для libcurl требуется линковка с `-lcurl`. Также libcurl зависит от других системных библиотек, но pkg-config позаботится об этом. Поэтому используем pkg-config.

```
# Makefile for Project 3 (HTML parser)
CC := gcc
CFLAGS := -Wall -Wextra -std=c11 -O2 $(shell pkg-config --cflags libcurl)
LDFLAGS :=
LDLIBS := $(shell pkg-config --libs libcurl)

TARGET := htmlparser
SRC := main.c htmlutils.c

all: $(TARGET)

$(TARGET): $(SRC:.c=.o)
    $(CC) $(LDFLAGS) $^ -o $@ $(LDLIBS)

%.o: %.c
    $(CC) $(CFLAGS) -DUSE_CURL -c $< -o $@

clean:
    $(RM) *.o $(TARGET)
```

Здесь ключевой момент: в CFLAGS и LDLIBS мы вставили вывод pkg-config для libcurl. Это автоматически добавит флаг `-DUSE_CURL`? – Нет, обратите внимание, мы вручную добавили `-DUSE_CURL` в правило компиляции `%.o: %.c`. То есть при компиляции каждого `.c` мы определяем макрос `USE_CURL`, чтобы в коде активировались части curl. Можно было это и в CFLAGS прописать, но мы хотели явно показать.

Если вам недоступен интернет или libcurl, они могут убрать `-DUSE_CURL` и соответствующие строки – тогда программа будет работать только с локальными файлами.

Meson (meson.build). С Meson подключение libcurl делается через `dependency('libcurl')`.

```
project('proj3-html-parse', 'c', version: '1.0')
# Ищем libcurl
curl_dep = dependency('libcurl', required : false)

executable('htmlparser', ['main.c', 'htmlutils.c'],
    c_args : ['-DUSE_CURL'],
    dependencies : [ curl_dep ] )
```

Здесь мы пометили `required: false` для libcurl – на случай, если её нет, Meson все равно позволит собрать проект (но тогда код при запуске сообщит, что curl не поддерживается). Мы также передали флаг `-DUSE_CURL` через `c_args` (можно было обернуть в условие `if curl_dep.found()` – но опустим для простоты). Meson автоматически подставит необходимые `-I` и `-lcurl` при наличии libcurl.

CMake (CMakeLists.txt):

```
cmake_minimum_required(VERSION 3.15)
project(proj3-html-parser C)

add_executable(htmlparser main.c htmlutils.c)

# Пытаемся найти libcurl
find_package(CURL)
```

```

if(CURL_FOUND)
    target_compile_definitions(htmlparser PRIVATE USE_CURL)
    target_include_directories(htmlparser PRIVATE ${CURL_INCLUDE_DIRS})
    target_link_libraries(htmlparser PRIVATE ${CURL_LIBRARIES})
else()
    message(WARNING "libcurl not found, building without network support.")
endif()

```

В CMake модуль FindCURL обычно идёт из коробки. Если найдено, мы определяем макрос USE_CURL и добавляем include-dir и библиотеки (CURL_LIBRARIES обычно равен `-lcurl` + системные). Если не найдено – просто предупреждаем. Такой CMakeLists.txt даёт представление, как обрабатывать опциональные зависимости.

Проект 4: Линейная алгебра (векторы и матрицы)

Описание. Четвёртый проект завершает блок плоской структуры. Тематика – основные вычисления линейной алгебры: операции над векторами и матрицами. В учебных целях можно реализовать небольшой модуль, умеющий:

- Считать из файла матрицу (или две) и выполнить операцию (например, умножение матриц, вычисление определителя, решение системы линейных уравнений методом Гаусса и т.п.).
- Работа с векторами: например, вычисление скалярного произведения, нормы, сортировка набора векторов по длине и т.д.

Чтобы не усложнять, возьмём пример: программа читает две матрицы A и B из файла и вычисляет их произведение $C = A * B$, результат сохраняет/печатает. Размерность матриц ограничим, скажем, 10×10 , и предполагаем совместимость для умножения (число столбцов A равно числу строк B). Этот проект отрабатывает работу с двумерными массивами, динамическое выделение памяти (`malloc` для матриц), вложенные циклы.

Структура (плоская). Назовём программу `linmath`.

```

proj4/
├─ main.c          # код, который парсит ввод, вызывает операции
├─ linalg.c        # реализация операций (умножение матриц, и др.)
├─ linalg.h
├─ Makefile
├─ meson.build
└─ CMakeLists.txt

```

Примеры кода:

`linalg.h` – интерфейс модуля `линалг`:

```

// linalg.h
#ifndef LINALG_H
#define LINALG_H

// Функция для умножения матриц.
// A размером r1 x c1, B размером c1 x c2; результат C размером r1 x c2 (должен быть выделен
//   заранее).
void mat_multiply(double **A, double **B, double **C, int r1, int c1, int c2);

// Функция для выделения памяти под матрицу r x c
double **alloc_matrix(int r, int c);

// Освобождение памяти матрицы
void free_matrix(double **M, int r);

#endif

```

linalg.c – реализация:

```
// linalg.c
#include "linalg.h"
#include <stdlib.h>

void mat_multiply(double **A, double **B, double **C, int r1, int c1, int c2) {
    for(int i=0; i<r1; ++i) {
        for(int j=0; j<c2; ++j) {
            C[i][j] = 0.0;
            for(int k=0; k<c1; ++k) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

double **alloc_matrix(int r, int c) {
    double **M = malloc(r * sizeof(double*));
    if (!M) return NULL;
    for(int i=0; i<r; ++i) {
        M[i] = malloc(c * sizeof(double));
        if(!M[i]) { /* handle allocation error, not shown for brevity */ }
    }
    return M;
}

void free_matrix(double **M, int r) {
    if (!M) return;
    for(int i=0; i<r; ++i) {
        free(M[i]);
    }
    free(M);
}
```

main.c – здесь читаем матрицы и используем функции:

```
// main.c
#include "linalg.h"
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if(argc < 2) {
        fprintf(stderr, "Usage: %s <matrix_file>\n", argv[0]);
        return 1;
    }
    FILE *f = fopen(argv[1], "r");
    if(!f) { perror("fopen"); return 1; }
    int r1, c1, r2, c2;
    // ожидаем в файле: размеры первой матрицы, потом сами элементы, затем размеры второй и
    // ↪ элементы
    if(fscanf(f, "%d %d", &r1, &c1) != 2) { fprintf(stderr, "Invalid format\n"); return 1; }
    double **A = alloc_matrix(r1, c1);
```

```

    for(int i=0; i<r1; ++i) {
        for(int j=0; j<c1; ++j) {
            fscanf(f, "%lf", &A[i][j]);
        }
    }
    if(fscanf(f, "%d %d", &r2, &c2) != 2) { fprintf(stderr, "Invalid format\n"); return 1; }
    double **B = alloc_matrix(r2, c2);
    for(int i=0; i<r2; ++i) {
        for(int j=0; j<c2; ++j) {
            fscanf(f, "%lf", &B[i][j]);
        }
    }
    fclose(f);

    if(c1 != r2) {
        fprintf(stderr, "Matrix dimensions incompatible for multiplication\n");
        return 1;
    }
    double **C = alloc_matrix(r1, c2);
    mat_multiply(A, B, C, r1, c1, c2);

    // Вывод результата
    printf("Result matrix (%dx%d):\n", r1, c2);
    for(int i=0; i<r1; ++i) {
        for(int j=0; j<c2; ++j) {
            printf("%8.2f ", C[i][j]);
        }
        printf("\n");
    }

    // освобождаем память
    free_matrix(A, r1);
    free_matrix(B, r2);
    free_matrix(C, r1);
    return 0;
}

```

Makefile. Здесь внешний библиотек нет, только `-lm` если бы использовались функции `math` (но у нас все сами). Однако, чтобы продемонстрировать, можем включить `-lm` на всякий случай (например, если бы считали определитель через `fabs` или что-то).

```

# Makefile for Project 4 (Linear Algebra)
CC := gcc
CFLAGS := -Wall -Wextra -std=c11 -O2
LDLIBS := -lm # на случай, если понадобятся математические функции
TARGET := linmath
SRC := main.c linalg.c

all: $(TARGET)

$(TARGET): $(SRC:.c=.o)
    $(CC) $^ -o $@ $(LDLIBS)

%.o: %.c

```

```
$(CC) $(CFLAGS) -c $< -o $@
```

```
clean:
```

```
$(RM) *.o $(TARGET)
```

Meson (meson.build):

```
project('proj4-linear-algebra', 'c', version: '1.0')
executable('linmath', ['main.c', 'linalg.c'])
```

Здесь настолько просто, что даже нечего комментировать – Meson сам всё сделает. По умолчанию он подтянет `-lm` если надо, хотя, возможно, и нет. Если действительно нужна мат. библиотека, можно явно указать `dependencies : [ dependency('m') ]` или `c_args: '-lm'`. Но Meson обычно на Linux не требует явно указывать `-lm` для стандартных функций? (На самом деле, для `sqrt` нужно `-lm`, Meson этого не знает сам, поэтому лучше добавить: `link_language : 'c', link_with : cc.find_library('m')` – но это лишняя сложность. В данном примере мы можем считать, что математические функции не использовались, либо добавить `-lm` в `LD_FLAGS` Makefile как мы сделали).

CMake (CMakeLists.txt):

```
cmake_minimum_required(VERSION 3.10)
project(proj4-linear-algebra C)
add_executable(linmath main.c linalg.c)
# линковка с math
target_link_libraries(linmath PRIVATE m)
```

Опять же, стандартно. Этот проект не приносит ничего нового в плане библиотек, но закрепляет основы CMake/Meson.

Проекты с иерархической структурой каталогов

В следующих проектах мы усложняем требования: проекты имеют **стандартную иерархию директорий**: исходники в `src/`, заголовки в `include/`, а компиляция производится с помещением объектных файлов в `obj/` и финальных программ в `bin/`. Makefile будет более сложным, поскольку нужно указать пути для объектов и создать директории. Зато код структуры станет чище, и мы подключим дополнительные внешние библиотеки (SDL2, GSL, etc.), расширяя функциональность проектов 1-4.

Проект 5: Графика на SDL2

Описание. Этот проект демонстрирует базовую работу с мультимедийной библиотекой **SDL2** (Simple DirectMedia Layer). SDL2 позволяет создавать окно, рисовать в нём, обрабатывать ввод. Можно реализовать простейшую графическую программу – например, отрисовку движущегося геометрического объекта или визуализацию математической функции. Для учебного примера возьмём: открыть окно 640x480, заполнить его фоном и нарисовать движущийся шар (кружок) – своего рода минимальная анимация. Это познакомит вас с инициализацией SDL (`SDL_Init`), созданием окна и `renderer`, основным циклом обработки событий (`event loop`) и рендерингом (`SDL_RenderClear`, `SDL_RenderFillRect` или `SDL_RenderDrawLine` etc.).

Структура (иерархическая):

```
proj5/
├── src/
│   ├── main.c           # код с функцией main (инициализация SDL, цикл)
│   ├── graphics.c       # вспомогательные функции рисования (например, рисовать шар)
│   └── graphics.h
├── include/             # заголовочные файлы
│   └── graphics.h       # (можно разместить здесь вместо src, если принято отделять интерфейсы)
├── obj/                 # будет содержать .o после компиляции
└── bin/                 # сюда помещаем итоговый исполняемый файл (например, bin/proj5_sdl)
```

```
└─ Makefile
└─ meson.build
└─ CMakeLists.txt
```

Примечание: В данной структуре `graphics.h` указан и в `src/`, и в `include/`. Реально файл один, можно решить хранить его в `include/` (лучше) и в `src/` не дублировать. Но для примера не критично – главное, что он отделён от `main.c`.

Пример кода:

`graphics.h` (поместим его в `include/`):

```
// graphics.h
#ifndef GRAPHICS_H
#define GRAPHICS_H

#include <SDL2/SDL.h>

// Функция для отрисовки круга (примитивно: заполняем квадратиками)
void draw_circle(SDL_Renderer *ren, int cx, int cy, int radius);

#endif
```

`graphics.c`:

```
#include "graphics.h"
#include <math.h>

void draw_circle(SDL_Renderer *ren, int cx, int cy, int radius) {
    // Простейший алгоритм: рисуем точками по окружности
    for(int w = 0; w < radius * 2; w++) {
        for(int h = 0; h < radius * 2; h++) {
            int dx = radius - w; // горизонтальное расстояние от центра
            int dy = radius - h; // вертикальное расстояние
            if(dx*dx + dy*dy <= radius * radius) {
                SDL_RenderDrawPoint(ren, cx + dx, cy + dy);
            }
        }
    }
}
```

`main.c`:

```
#include <SDL2/SDL.h>
#include "graphics.h"
#include <stdio.h>
#include <stdbool.h>

int main(int argc, char *argv[]) {
    if (SDL_Init(SDL_INIT_VIDEO) != 0) {
        fprintf(stderr, "SDL_Init Error: %s\n", SDL_GetError());
        return 1;
    }
    SDL_Window *win = SDL_CreateWindow("Moving Circle",
                                       SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED,
                                       640, 480, SDL_WINDOW_SHOWN);

    if (!win) {
        fprintf(stderr, "SDL_CreateWindow Error: %s\n", SDL_GetError());
    }
}
```

```

    SDL_Quit();
    return 1;
}
SDL_Renderer *ren = SDL_CreateRenderer(win, -1, SDL_RENDERER_ACCELERATED);
if (!ren) {
    fprintf(stderr, "SDL_CreateRenderer Error: %s\n", SDL_GetError());
    SDL_DestroyWindow(win);
    SDL_Quit();
    return 1;
}
bool running = true;
int x = 0, y = 240, vx = 2; // позиция и скорость кружка
SDL_Event e;
while(running) {
    // обработка событий
    while(SDL_PollEvent(&e)) {
        if(e.type == SDL_QUIT) {
            running = false;
        }
    }
    // логика движения
    x += vx;
    if(x - 50 > 640) { x = -50; } // цикл движения: вылетел справа - появится слева
    // отрисовка
    SDL_SetRenderDrawColor(ren, 0, 0, 0, 255); // чёрный фон
    SDL_RenderClear(ren);
    SDL_SetRenderDrawColor(ren, 255, 0, 0, 255); // красный цвет для круга
    draw_circle(ren, x, y, 50);
    SDL_RenderPresent(ren);
    SDL_Delay(10); // небольшая задержка ~100 FPS
}
SDL_DestroyRenderer(ren);
SDL_DestroyWindow(win);
SDL_Quit();
return 0;
}

```

Эта программа рисует красный круг радиусом 50, движущийся по горизонтали. Очень простой механизм.

Makefile. Здесь проект иерархический, поэтому Makefile усложняется: нужно создать папки `obj/` и `bin/` при сборке. Будем компилировать объектики в `obj`, а линковать в `bin`.

Кроме того, подключается SDL2 – нужно использовать `sdl2-config` или `pkg-config`. SDL2 предоставляет `sdl2-config` утилиту, но `pkg-config` тоже работает (`sdl2.pc`). Мы воспользуемся `pkg-config` как современным подходом.

```

# Makefile for Project 5 (SDL2 Graphics) - hierarchical structure
CC := gcc
CFLAGS := -Wall -Wextra -std=c11 -O2 $(shell pkg-config --cflags sdl2)
LDLIBS := $(shell pkg-config --libs sdl2)

SRC_DIR := src
OBJ_DIR := obj
BIN_DIR := bin
TARGET := $(BIN_DIR)/proj5_sdl

```

```

SRC := $(wildcard $(SRC_DIR)/*.c)
OBJ := $(patsubst $(SRC_DIR)/%.c, $(OBJ_DIR)/%.o, $(SRC))

all: $(BIN_DIR) $(OBJ_DIR) $(TARGET)

$(BIN_DIR) $(OBJ_DIR):
    mkdir -p $@

$(TARGET): $(OBJ)
    $(CC) $^ -o $@ $(LDLIBS)

# Правило компиляции каждого .c -> .o (с указанием путей)
$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
    $(CC) $(CFLAGS) -Iinclude -c $< -o $@

clean:
    $(RM) -r $(OBJ_DIR) $(BIN_DIR)

```

Пояснения:

- SRC := \$(wildcard src/*.c) собирает все C-файлы в src.
- Используем шаблон для OBJ, заменяя src/ на obj/ и .c на .o.
- Цель \$(BIN_DIR) и \$(OBJ_DIR) создаёт каталоги, если их нет.
- Компиляция: важно указать -Iinclude, чтобы компилятор искал заголовки в папке include (например, graphics.h).
- Линковка: кладем результат в bin/proj5_sdl. В LDLIBS добавлены флаги SDL2 через pkg-config (это превратится в -lSDL2 -D_REENTRANT ... и возможно -lpthread автоматически).
- clean удаляет папки целиком.

Meson (meson.build). В Meson можно напрямую задать папки, но по умолчанию Meson тоже делает похожее: obj хранятся внутри builddir автоматически, binагики тоже там. Однако мы можем указать при инсталляции, куда класть. Для простоты, напишем один meson.build в корне:

```

project('proj5-sdl2-graphics', 'c', version: '1.0')
sdl_dep = dependency('sdl2', required: true)

# Собираем все .c из src/
src_files = files('src/main.c', 'src/graphics.c')
executable('proj5_sdl', src_files,
    include_directories: include_directories('include'),
    dependencies: [ sdl_dep ])

```

Meson сам не требует указывать objdir – он хранит объекты в своём builddir. Мы просто перечисляем исходники с путями, указываем, где искать includes, и подключаем dependency SDL2. Meson позаботится о создании нужных поддиректорий в build/.

CMake (CMakeLists.txt):

```

cmake_minimum_required(VERSION 3.14)
project(proj5_sdl2 C)
# Ищем SDL2
find_package(SDL2 REQUIRED)
include_directories(${SDL2_INCLUDE_DIRS})

file(GLOB SRC_FILES "src/*.c")
add_executable(proj5_sdl ${SRC_FILES})
target_include_directories(proj5_sdl PRIVATE ${CMAKE_SOURCE_DIR}/include)

```



```
target_link_libraries(proj5_sdl PRIVATE ${SDL2_LIBRARIES})
```

Здесь:

- Используем `file(GLOB ...)` для удобства собрать все `.c` файлы – обычно CMake не рекомендует `GLOB` для стабильности, но в учебном примере можно.
- `find_package(SDL2 REQUIRED)` найдёт SDL2, обычно предоставляя `SDL2_INCLUDE_DIRS` и `SDL2_LIBRARIES`. Если CMake версии $\geq 2.8.12$, то может быть даже `SDL2 : SDL2 target`. Но чтобы не усложнять, пользуемся старым стилем переменных.
- Добавляем `include path include` папки и линкуем с SDL2.

Проект 6: Использование GSL для численного анализа

Описание. В этом проекте обращаемся к библиотеке **GSL (GNU Scientific Library)** для решения задач, которые сложно или долго реализовывать вручную. GSL предоставляет богатый набор алгоритмов: от статистики до решения дифференциальных уравнений. Для первого курса можно взять что-то попроще: например, использование генератора случайных чисел и вычисление статистических характеристик выборки, или решение системы линейных уравнений, или вычисление интеграла функции численно.

Чтобы связать с предыдущим опытом (проект 4), можно, например, использовать GSL для нахождения **обратной матрицы** или **решения линейной системы**. Или показать вычисление специальных функций (Bessel, Gamma – есть в GSL) по сравнению со стандартной библиотекой.

Конкретный пример: программа генерирует большой массив случайных чис с нормальным распределением (с помощью GSL RNG), затем вычисляет среднее и дисперсию (используя функции GSL Statistics) и сравнивает с теоретическими значениями. Это продемонстрирует работу и с генератором, и со статистическими функциями библиотеки.

Структура. иерархическая, назовём программу `gsl_stats`.

```
proj6/
├── src/
│   ├── main.c          # основная функция: генерация, вызов функций GSL, вывод
│   └── analysis.c      # можно вынести часть логики (например, функцию для печати статистики)
├── include/
│   └── analysis.h
├── obj/
├── bin/
├── Makefile
├── meson.build
└── CMakeLists.txt
```

Примеры кода:

`analysis.h:`

```
#ifndef ANALYSIS_H
#define ANALYSIS_H
```

```
void print_statistics(const double data[], size_t n);
```

```
#endif
```

`analysis.c:`

```
#include "analysis.h"
#include <gsl/gsl_statistics.h>
#include <stdio.h>
```

```
void print_statistics(const double data[], size_t n) {
```

```

    double mean = gsl_stats_mean(data, 1, n);
    double var = gsl_stats_variance(data, 1, n);
    double max = gsl_stats_max(data, 1, n);
    double min = gsl_stats_min(data, 1, n);
    printf("Count = %zu\n", n);
    printf("Mean = %g\n", mean);
    printf("Variance = %g\n", var);
    printf("Min = %g, Max = %g\n", min, max);
}

```

Здесь мы используем функции GSL Statistics: они принимают массив, stride (шаг 1) и размер, возвращают соответствующую статистику.

main.c:

```

#include <gsl/gsl_rng.h>
#include <gsl/gsl_randist.h>
#include <stdio.h>
#include <stdlib.h>
#include "analysis.h"

int main(int argc, char *argv[]) {
    size_t N = 1000000;
    if(argc > 1) {
        N = strtoull(argv[1], NULL, 10);
        if(N == 0) N = 1000000;
    }
    // Инициализация генератора случайных чис
    const gsl_rng_type * T;
    gsl_rng *r;
    gsl_rng_env_setup(); // настройка через переменные окружения, если заданы
    T = gsl_rng_default;
    r = gsl_rng_alloc(T);
    // Генерируем выборку
    double *data = malloc(N * sizeof(double));
    for(size_t i = 0; i < N; ++i) {
        // случайные числа по нормальному распределению (mean=0, sigma=1)
        data[i] = gsl_ran_gaussian(r, 1.0);
    }
    // Выводим несколько первых значений (для примера)
    printf("First 5 values: ");
    for(size_t i=0; i<5 && i<N; ++i) {
        printf("%.3f ", data[i]);
    }
    printf("...\n");
    // Вычисляем статистики
    print_statistics(data, N);
    // Освобождаем
    free(data);
    gsl_rng_free(r);
    return 0;
}

```

Эта программа генерирует N (по умолчанию 1e6) случайных чис с нормальным распределением (0,1) используя GSL, а потом печатает выборочные статистики. GSL здесь демонстрируется в части генератора (gsl_rng) и статистики (gsl_

stats).

Makefile. Этот проект требует линковки с **GSL** (и её зависимостью CBLAS, а также, возможно, -lm). Мы воспользуемся pkg-config, если доступно, или укажем явно. Для надёжности можно явно: -lgsl -lgslcblas -lm.

```
# Makefile for Project 6 (GSL usage) - hierarchical
CC := gcc
CFLAGS := -Wall -Wextra -std=c11 -O2 $(shell pkg-config --cflags gsl 2>/dev/null)
LDLIBS := $(shell pkg-config --libs gsl 2>/dev/null)
# Если pkg-config не нашёл gsl, используем стандартные флаги:
ifeq ($(LDLIBS),)
    LDLIBS := -lgsl -lgslcblas -lm
endif

SRC_DIR := src
OBJ_DIR := obj
BIN_DIR := bin
TARGET := $(BIN_DIR)/gsl_stats
SRC := $(wildcard $(SRC_DIR)/*.c)
OBJ := $(patsubst $(SRC_DIR)/%.c, $(OBJ_DIR)/%.o, $(SRC))

all: $(OBJ_DIR) $(BIN_DIR) $(TARGET)

$(OBJ_DIR) $(BIN_DIR):
    mkdir -p $@

$(TARGET): $(OBJ)
    $(CC) $^ -o $@ $(LDLIBS)

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
    $(CC) $(CFLAGS) -Iinclude -c $< -o $@

clean:
    $(RM) -r $(OBJ_DIR) $(BIN_DIR)
```

Объяснение: Пытаемся через pkg-config (pkg-config --libs gsl) получить флаги. Если pkg-config не настроен для GSL (в некоторых системах), тогда вручную задаём -lgsl -lgslcblas -lm.

Meson (meson.build):

```
project('proj6-gsl', 'c', version: '1.0')
gsl_dep = dependency('gsl', required: true)
executable('gsl_stats', ['src/main.c', 'src/analysis.c'],
    include_directories: include_directories('include'),
    dependencies: [ gsl_dep ])
```

Если GSL установлена, Meson через pkg-config её найдёт. Мы указали include и зависимости. Этого хватит, Meson сам определит -lgsl -lgslcblas -lm.

CMake (CMakeLists.txt):

```
cmake_minimum_required(VERSION 3.10)
project(proj6_gsl C)
find_package(GSL REQUIRED)
add_executable(gsl_stats src/main.c src/analysis.c)
target_include_directories(gsl_stats PRIVATE ${CMAKE_SOURCE_DIR}/include)
target_link_libraries(gsl_stats PRIVATE GSL::gsl GSL::gslcblas)
```

Здесь важно: современный FindGSL.cmake (с CMake 3.7+) определяет IMPORTED targets `GSL::gsl` и `GSL::gslcblas`, которыми удобно пользоваться. Мы их и используем для линковки. Если CMake старый, можно было бы `target_link_libraries(... ${GSL_LIBRARIES})`, но возьмём новый стиль.

Проект 7: Расширенное сокетное взаимодействие (многопоточный сервер)

Описание. Возвращаемся к теме сетей (проект 2), но добавляем сложности. “Расширенное” взаимодействие может значить:

- поддержка **нескольких одновременных клиентов** (конкурентный сервер). Это потребует использования многопоточности (pthread) или неблокирующего ввода-вывода (select/epoll). Для учебного примера более реалистично показать многопоточность: каждый клиент обслуживается в отдельном потоке.
- поддержка какого-то протокола более высокого уровня. Например, реализовать простой чат-сервер, куда клиенты могут отправлять сообщения, а сервер рассылает их всем подключенным (broadcast). Это потребует управления списком клиентов и синхронизации потоков (mutex).
- либо интеграция готовых библиотек для сети, например, использовать **libcurl**/HTTP клиент, но это мы уже затронули в проекте 3.

Оптимально: сделать многопоточный эхо-сервер, как расширение проекта 2. Клиенты могут подключаться одновременно, сервер создаёт поток на каждое подключение. Это научит:

- Работе с **pthread** (создание потоков, возможно, pthread_detach).
- Обеспечению потокобезопасности при доступе к общим ресурсам (в простом эхо их нет, но если был бы счетчик клиентов – нужен мьютекс).
- Правильному завершению (например, закрытие сокета, завершение потока).

Структура. иерархическая. Можно использовать код из проекта 2, но переработать под многопоточность и убрать явную программу клиента (в принципе, можно использовать клиент из проекта 2 для теста). Или включить здесь и клиент, но это не обязательно; сфокусируемся на сервере.

```
proj7/
├── src/
│   ├── server.c      # многопоточный сервер
│   ├── server.h      # (если есть общие структуры/функции)
│   ├── client.c      # (опционально клиент, или можно переиспользовать proj2/client)
│   ├── net_utils.c   # можно взять из proj2, модифицировать если нужно
│   └── net_utils.h
├── include/
│   └── (можно хранить server.h, net_utils.h здесь)
├── obj/
├── bin/
├── Makefile
├── meson.build
└── CMakeLists.txt
```

Допустим, мы включаем клиент тоже, но основной акцент – сервер.

Примеры кода (фрагменты):

server.c:

```
#include "net_utils.h"
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

#define PORT 9090
```

```

void *handle_client(void *arg) {
    int client_fd = *(int*)arg;
    free(arg);
    printf("Client thread %lu started\n", pthread_self());
    char buf[512];
    ssize_t n;
    // эхо-обработка
    while((n = recv(client_fd, buf, sizeof(buf), 0)) > 0) {
        send(client_fd, buf, n, 0);
    }
    close(client_fd);
    printf("Client thread %lu finished\n", pthread_self());
    return NULL;
}

int main() {
    int server_fd = create_server_socket(PORT);
    if(server_fd < 0) {
        return 1;
    }
    printf("Multi-threaded Server listening on %d\n", PORT);
    while(1) {
        int client_fd = accept(server_fd, NULL, NULL);
        if(client_fd < 0) {
            perror("accept");
            break;
        }
        // каждый раз, когда клиент подключается, создаём поток
        pthread_t tid;
        int *pclient = malloc(sizeof(int));
        *pclient = client_fd;
        if(pthread_create(&tid, NULL, handle_client, pclient) != 0) {
            perror("pthread_create");
            close(client_fd);
            free(pclient);
        } else {
            pthread_detach(tid); // отделяем поток, чтобы не ждать join
        }
    }
    close(server_fd);
    return 0;
}

```

net_utils.c и net_utils.h можно взять из проекта 2 без изменений (создание серверного сокета etc.). Ключевая часть – использование pthread_create для обслуживания клиента.

Makefile. Нам понадобится флаг -pthread при компиляции и линковке. Кроме того, структура каталогов. Библиотеки внешние: здесь только pthread, но pthread на Linux подключается через -pthread (что добавляет -lpthread и включает флаги для компилятора).

```

# Makefile for Project 7 (Multithreaded Sockets) - hierarchical
CC := gcc
CFLAGS := -Wall -Wextra -std=c11 -O2 -pthread
LDLIBS := -pthread

```

```

SRC_DIR := src
OBJ_DIR := obj
BIN_DIR := bin
TARGETS := $(BIN_DIR)/server_mt $(BIN_DIR)/client_mt

SRC_SERVER := server.c net_utils.c
SRC_CLIENT := client.c net_utils.c
OBJ_SERVER := $(addprefix $(OBJ_DIR)/, $(SRC_SERVER:.c=.o))
OBJ_CLIENT := $(addprefix $(OBJ_DIR)/, $(SRC_CLIENT:.c=.o))

all: $(OBJ_DIR) $(BIN_DIR) server_mt client_mt

$(OBJ_DIR) $(BIN_DIR):
    mkdir -p $@

server_mt: $(BIN_DIR)/server_mt
client_mt: $(BIN_DIR)/client_mt

$(BIN_DIR)/server_mt: $(OBJ_SERVER)
    $(CC) $^ -o $@ $(LDLIBS)

$(BIN_DIR)/client_mt: $(OBJ_CLIENT)
    $(CC) $^ -o $@ $(LDLIBS)

# Общий шаблон для компиляции .c в obj
$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
    $(CC) $(CFLAGS) -Iinclude -c $< -o $@

clean:
    $(RM) -r $(OBJ_DIR) $(BIN_DIR)

```

Здесь `server_mt` и `client_mt` – названия для серверного и клиентского приложений. Мы указали их как цели. Можно было разделить Makefile на две части, но мы сделали вместе.

Meson (meson.build):

```

project('proj7-advanced-sockets', 'c', version: '1.0')
# pthread в Meson на Linux можно не указывать явно, но добавим для переносимости
thread_dep = dependency('threads')

# Собираем исполняемые
executable('server_mt', ['src/server.c', 'src/net_utils.c'],
    include_directories: include_directories('include'),
    dependencies: [ thread_dep ])

executable('client_mt', ['src/client.c', 'src/net_utils.c'],
    include_directories: include_directories('include'),
    dependencies: [ thread_dep ])

```

Meson имеет встроенный `support dependency('threads')`, который на Unix даст pthread флаг, на Windows – ничего или аналог, так что лучше добавить. Остальное аналогично проекту 2.

CMake (CMakeLists.txt):

```
cmake_minimum_required(VERSION 3.5)
```

```

project(proj7_adv_sockets C)
find_package(Threads REQUIRED)
add_executable(server_mt src/server.c src/net_utils.c)
add_executable(client_mt src/client.c src/net_utils.c)
target_include_directories(server_mt PRIVATE ${CMAKE_SOURCE_DIR}/include)
target_include_directories(client_mt PRIVATE ${CMAKE_SOURCE_DIR}/include)
target_link_libraries(server_mt PRIVATE Threads::Threads)
target_link_libraries(client_mt PRIVATE Threads::Threads)

```

`find_package(Threads)` – стандартный для pthread. Он добавляет `Threads::Threads` target.

Проект 8: Парсинг форматов JSON или XML

Описание. Последний проект знакомит с разборами данных в форматах **JSON** и **XML** – распространённые задачи при обмене данными и конфигурациях. Можно выбрать один из форматов или предложить оба, но для учебного примера достаточно что-то одно продемонстрировать.

Например, реализовать программу, которая читает JSON-файл с каким-то содержимым (скажем, массив объектов с полями `name` и `value`) и выводит какие-то агрегированные данные. Аналогично для XML: прочитать XML-документ и выбрать определённые узлы. Ручной парсинг таких структур сложен, поэтому обязательно использовать библиотеки:

- Для JSON на C есть популярная легковесная библиотека **cJSON** (заголовочный + исходник, можно включить в проект) или **json-c** (библиотека с `pkg-config`).
- Для XML – классика **libxml2** (`pkg-config libxml-2.0`) либо **tinycl2** (C++ though).

Возьмём JSON с cJSON для примера. Скажем, JSON файл `data.json` содержит список студентов:

```

{
  "group": "CS-101",
  "students": [
    {"name": "Alice", "age": 20},
    {"name": "Bob", "age": 22}
  ]
}

```

Наша программа прочитает файл, распарсит JSON и выведет, сколько студентов и их имена.

Структура. иерархическая. Добавим библиотеку cJSON прямо в проект для наглядности (cJSON распространяется как один `.c` и `.h`).

```

proj8/
├── src/
│   ├── main.c           # код: загрузка файла, парсинг JSON, вывод результата
│   ├── parser.c         # код: функции, использующие cJSON (или libxml)
│   ├── cJSON.c          # исходник библиотеки cJSON (включён в проект)
│   └── cJSON.h
├── include/
│   └── parser.h          # объявление функций парсера, если нужно
├── obj/
├── bin/
├── Makefile
├── meson.build
└── CMakeLists.txt

```

Примеры кода:

`parser.h` (если нужен):

```

#ifndef PARSER_H
#define PARSER_H

int load_and_parse(const char *filename);

#endif

parser.c:

#include "cJSON.h"
#include <stdio.h>
#include <stdlib.h>

int load_and_parse(const char *filename) {
    // Читаем файл полностью в память
    FILE *f = fopen(filename, "r");
    if(!f) { perror("fopen"); return -1; }
    fseek(f, 0, SEEK_END);
    long len = ftell(f);
    fseek(f, 0, SEEK_SET);
    char *data = malloc(len + 1);
    if(!data) { fclose(f); return -1; }
    fread(data, 1, len, f);
    data[len] = '\0';
    fclose(f);

    // Парсим JSON
    cJSON *json = cJSON_Parse(data);
    free(data);
    if(!json) {
        fprintf(stderr, "JSON parse error before: %s\n", cJSON_GetErrorPtr());
        return -1;
    }
    // Предположим, верхний уровень – объект с полем "students" (массив)
    cJSON *students = cJSON_GetObjectItem(json, "students");
    if(!cJSON_IsArray(students)) {
        fprintf(stderr, "\"students\" is not an array\n");
        cJSON_Delete(json);
        return -1;
    }
    int count = cJSON_GetArraySize(students);
    printf("Number of students: %d\n", count);
    for(int i = 0; i < count; ++i) {
        cJSON *stu = cJSON_GetArrayItem(students, i);
        cJSON *name = cJSON_GetObjectItem(stu, "name");
        cJSON *age = cJSON_GetObjectItem(stu, "age");
        if(cJSON_IsString(name) && cJSON_IsNumber(age)) {
            printf("Student %d: %s, age %d\n", i+1, name->valuestring, age->valueint);
        }
    }
    cJSON_Delete(json);
    return 0;
}

```



```
main.c:
#include <stdio.h>
#include "parser.h"

int main(int argc, char *argv[]) {
    if(argc < 2) {
        fprintf(stderr, "Usage: %s <json_file>\n", argv[0]);
        return 1;
    }
    if(load_and_parse(argv[1]) != 0) {
        fprintf(stderr, "Failed to parse %s\n", argv[1]);
        return 1;
    }
    return 0;
}
```

cJSON.c/cJSON.h: привели бы их, но они достаточно большие. Предполагается, что мы их скачали и добавили в проект (cJSON – open source, MIT License).

Makefile. Особенность: здесь мы включили cJSON source *внутри проекта*, так что просто компилируем его вместе с остальными .c. Если бы мы использовали системную библиотеку (json-c или libxml2), тогда было бы через pkg-config. В Makefile покажем вариант с встроенной cJSON.

```
# Makefile for Project 8 (JSON parsing) - hierarchical
CC := gcc
CFLAGS := -Wall -Wextra -std=c11 -O2
LDLIBS := # нет внешних .so, cJSON интегрирован как исходник

SRC_DIR := src
OBJ_DIR := obj
BIN_DIR := bin
TARGET := $(BIN_DIR)/json_parser

# собираем все .c кроме, допустим, cJSON.c, хотя мы его тоже скомпилируем
SRC := $(wildcard $(SRC_DIR)/*.c)
OBJ := $(patsubst $(SRC_DIR)/%.c, $(OBJ_DIR)/%.o, $(SRC))

all: $(OBJ_DIR) $(BIN_DIR) $(TARGET)

$(OBJ_DIR) $(BIN_DIR):
    mkdir -p $@

$(TARGET): $(OBJ)
    $(CC) $^ -o $@ $(LDLIBS)

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
    $(CC) $(CFLAGS) -Iinclude -c $< -o $@

clean:
    $(RM) -r $(OBJ_DIR) $(BIN_DIR)
```

Если бы мы предпочли libxml2: тогда вместо cJSON.c/h, подключаем бы <libxml/parser.h> и линковали \$(shell pkg-config --cflags --libs libxml-2.0). Можно изучить упомянуть оба способа.

Meson (meson.build):

```
project('proj8-json-xml-parse', 'c', version: '1.0')
# Здесь вариант с cJSON встроенным
executable('json_parser', ['src/main.c', 'src/parser.c', 'src/cJSON.c'],
    include_directories: include_directories('include'))
```

Если бы libxml2: xml_dep = dependency('libxml-2.0') и добавить в dependencies.

CMake (CMakeLists.txt):

```
cmake_minimum_required(VERSION 3.5)
project(proj8_parse C)
add_executable(json_parser src/main.c src/parser.c src/cJSON.c)
target_include_directories(json_parser PRIVATE ${CMAKE_SOURCE_DIR}/include)
# cJSON не требует специальных lib, если libxml2:
# find_package(LibXml2 REQUIRED) etc...
```

Этого достаточно для встроенной cJSON. Если хотим, можем добавить опционально find_package(LibXml2).

На этом все восемь проектов представлены. Вам следует изучить предложенные Makefile, meson.build, CMakeLists.txt и обратить внимание на:

- Различия в синтаксисе и подходе: Makefile использует правила и переменные, Meson – декларативный executable() и dependency(), CMake – директивы add_executable, target_link_libraries и find_package.
- Как организованы пути в иерархических проектах (флаги -Iinclude, шаблоны для файлов .o в подкаталогах).
- Как подключаются внешние библиотеки: через pkg-config (Makefile), dependency в Meson, find_package в CMake.
- Чем отличаются плоская и иерархическая структуры на практике: в плоских примерах Makefile проще, но файлы все вперемешку; в иерархических – Makefile сложнее, зато код структурированнее.
- Как добавление потоков влияет на флаги (-pthread) и необходимость специального указания в разных системах сборки (в Meson просто dependency('threads'), в CMake Threads::Threads).

Изучив (и самостоятельно доработав) эти примеры, вы можете овладеть базовыми навыками организации C-проекта и настройки его сборки под разные инструменты, что является важным шагом в становлении профессиональных навыков программирования.

